Instruções de Reprodução do Projeto

1. Objetivo deste documento

Este guia permite que qualquer pessoa reproduza o núcleo do coprocessador gráfico integrado ao HPS na placa Terasic DE1-SoC: abrir o projeto no Quartus, compilar (ou usar o .sof já gerado), programar a FPGA, bootar o Linux no HPS, compilar e executar os programas de teste via driver PIO, e, opcionalmente, explorar os cenários de demonstração descritos nos arquivos de teste.

2. Requisitos de hardware

- Placa Terasic DE1-SoC com FPGA Intel/Altera Cyclone V 5CSEMA5F31C6
- Monitor ou TV com entrada VGA e cabo VGA
- Fonte da placa e cabo USB-Blaster (USB para programação)
- Cabo Ethernet (recomendado) ou serial/USB para console do Linux no HPS
- Computador com portas USB (programação) e, preferencialmente, Ethernet

3. Requisitos de software

- Intel Quartus Prime (Lite ou Standard) — versão de referência: 25.1std.0 (ou 18.1/20.1/21.1 compatíveis com Cyclone V)
- Device support Cyclone V instalado no Quartus
- Driver USB-Blaster instalado (Windows/Linux)
- Toolchain ARM (gcc-arm-linux-gnueabihf ou equivalente) para compilar o software HPS
- Imagem de Linux para HPS (Angstrom/Ubuntu ou a fornecida no kit DE1-SoC)
- Arquivo do projeto: pasta PBL02_AFJ_Final/PBL_com_H/PBL_com_H/my_first_hps-fpga_base dentro do ZIP da versão final
- Arquivos de teste e driver: Arquivos para teste/ (isa_coproc.h, coproc_driver.h, coproc_driver.s, test_grafico_hps.c e pasta testes/)

4. Abrir o projeto no Quartus

Passo a passo:

- 1. Abra o Quartus Prime.
- 2. File → Open Project… e selecione o arquivo soc_system.qpf dentro da pasta my_first_hps-fpga_base.
- 3. Confirme que o top-level entity é ghrd_top (Assignments → Settings / Project Settings).
- 4. Confirme o device: 5CSEMA5F31C6 (Cyclone V).
- 5. Verifique em Assignments → Settings → Files se os arquivos RTL (rtl/*.v), o Qsys (soc_system.qsys) e os .qip principais estão listados (o .qsf da versão final já deve trazê-los).
- 6. O sistema Qsys (soc_system) integra o HPS, PIOs de comando/status e o núcleo do coprocessador gráfico.

5. Compilação

5.1 Compilar do zero

- Processing → Start Compilation (ou atalho Ctrl+L).
- Aguarde Analysis & Synthesis, Fitter, Assembler e Timing Analyzer.
- Ao final, o arquivo output_files/soc_system.sof deve ser gerado/atualizado.
- Consulte output_files/soc_system.fit.summary e soc_system.sta.summary para recursos e timing.

5.2 Usar o .sof já compilado (mais rápido)

A versão final já contém output_files/soc_system.sof. Se o device e a revisão forem os mesmos, pode pular a compilação e ir direto à programação (seção 6). Ainda assim, recomenda-se uma compilação local para validar o ambiente do laboratório.

6. Programar a FPGA

- 1. Ligue a DE1-SoC, conecte o USB-Blaster ao PC e o cabo VGA ao monitor.
- 2. No Quartus: Tools → Programmer.
- 3. Hardware Setup → selecione USB-Blaster (ou o cabo detectado).
- 4. Add File… → output_files/soc_system.sof
- 5. Marque Program/Configure e clique em Start.
- 6. Aguarde a programação concluir. O LED de configuração deve indicar sucesso.

Após a programação, o HPS pode ser bootado (cartão SD com Linux ou boot via UART/JTAG conforme a configuração do projeto).

7. Software no HPS — Driver e testes

7.1 Compilação do driver e do programa de teste

No ambiente Linux do HPS (ou cross-compile no host):

- Copie os arquivos isa_coproc.h, coproc_driver.h, coproc_driver.s e test_grafico_hps.c para o HPS.
- Compile: gcc -std=c99 -O2 -Wall -o test_grafico test_grafico_hps.c coproc_driver.s
- Execute com privilégios (acesso a /dev/mem): sudo ./test_grafico

O driver (coproc_driver.s) mapeia os PIOs de comando e status via /dev/mem. Não depende de chaves ou botões da placa: o modo fila, o passo a passo e o reset são comandos CTRL (opcode 62) enviados pelo software.

7.2 Interface do driver (resumo)

- coproc_init() / coproc_close() — abre/fecha mapeamento de memória
- coproc_send(instr) — enfileira e espera terminar
- coproc_enfileira(instr) / coproc_espera() — modo fila
- coproc_modo_fila(1|0), coproc_passo(), coproc_reset(0|1)
- coproc_status() — lê pio_status (busy, done, fifo, vsync, halt, buffers)

7.3 Cenários de teste principais

O programa test_grafico_hps.c e os arquivos em testes/ cobrem:

- Scroll absoluto e relativo do background (SET_BG_X/Y, MOV_BG)
- Sprites: posição, movimento, configuração (enable/flip/prioridade), padrão e hide
- Polígonos: desenho, tamanho, cor, movimento e hide (4 slots)
- Troca de tilemap (SWAP_BG) e frame-buffer (SWAP_FB) sincronizados com VSYNC
- SET_TILE e WR_RAM (programação de tilemaps e padrões)
- ALU (ADD, SUB, MOVI, MOV, AND, OR) e controle de fluxo (JMP, BEQ, HALT)
- Prioridade de camadas (SET_LAYER, SET_PRI) e modo fila / passo a passo

7.4 Sequência sugerida de demonstração

- 1. Programar a FPGA e bootar o Linux no HPS.
- 2. Compilar e executar o teste de inicialização (camadas ligadas, background visível).
- 3. Teste de scroll do background (absoluto + relativo).
- 4. Mostrar e mover sprites (SPR_POS, SPR_MOV, SPR_CFG, SPR_PAT).
- 5. Mostrar e mover polígonos (PLY_DRAW, PLY_ADJ, PLY_COR, PLY_MOV).
- 6. Demonstração de prioridades e transparência.
- 7. Programação de tiles (SET_TILE no tilemap oculto + SWAP_BG).
- 8. Testes de FIFO, HALT e reset por software.

8. Referências rápidas de módulos (RTL)

Núcleo ISA: control_unit.v, isa_core.v, alu.v, gpr_bank.v / banco_registradores.v, fifo_cmd.v.
Interface HPS: pio_cmd_iface.v (comandos e status via PIO).
Motores gráficos: motor_background.v, motor_sprites.v, rasterizador_multi.v, compositor.v.
Memórias: ram_tilemap.v / ram_tilemapB.v, ram_padrao_tiles.v, ram_sprites.v, frame_buffer_duplo.v.
VGA e clocks: vga_driver.v, meu_pll.v, pllpara100.v.
Top-level: ghrd_top.v (instancia soc_system + coprocessor_core).
ISA documentada em: ISA_Coprocessador_Grafico.pdf (31 instruções, formatos R/I/G/J).